
Lab Assignment# 11.1

Program	:B.Tech (CSE)
Specialization	:-
Course Title	:AI Assisted Coding
Course Code	:23CS002PC304
Semester	II
Academic Session	:2025-2026
Name of Student	:A.Nagasai
Enrollment No.	: 2403A51L18
Batch No	: 51
Date	:17/02/26

Task-1–Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample code:

class Stack:

 Pass

Prompt: generate a Stack class with push, pop, peek, and is_empty Methods and code.

Code:

```

1 class Stack:
2     def __init__(self):
3         """Initialize an empty stack."""
4         self.items = []
5
6     def push(self, item):
7         """Add an item to the top of the stack.
8
9         Args:
10             item: The item to be added to the stack.
11         """
12         self.items.append(item)
13
14     def pop(self):
15         """Remove and return the top item from the stack.
16
17         Returns:
18             The top item from the stack.
19
20         Raises:
21             IndexError: If the stack is empty.
22         """
23         if self.is_empty():
24             raise IndexError("Cannot pop from an empty stack")
25         return self.items.pop()
26
27     def peek(self):
28         """Return the top item from the stack without removing it.
29
30         Returns:
31             The top item from the stack.
32
33         Raises:
34             IndexError: If the stack is empty.
35         """
36         if self.is_empty():
37             raise IndexError("Cannot peek at an empty stack")
38         return self.items[-1]
39
40     def is_empty(self):
41         """Check if the stack is empty.
42
43         Returns:
44             True if the stack is empty, False otherwise.
45         """
46         return len(self.items) == 0
47
48     def size(self):
49         """Return the number of items in the stack.
50
51         Returns:
52             The number of items in the stack.
53         """
54         return len(self.items)
55
56     def __str__(self):
57         """Return a string representation of the stack.
58
59         Returns:
60             A string representation of the stack.
61         """
62         return f"Stack({self.items})"

```

```

63
64
65 # Example usage and testing
66 if __name__ == "__main__":
67     # Create a new stack
68     stack = Stack()
69
70     # Test is_empty on empty stack
71     print(f"Is stack empty? {stack.is_empty()}")
72     print(f"Stack size: {stack.size()}")
73     print()
74
75     # Push items onto the stack
76     print("Pushing items: 10, 20, 30")
77     stack.push(10)
78     stack.push(20)
79     stack.push(30)
80     print(f"Stack: {stack}")
81     print(f"Is stack empty? {stack.is_empty()}")
82     print(f"Stack size: {stack.size()}")
83     print()
84
85     # Peek at the top item
86     print(f"Peek at top: {stack.peak()}")
87     print(f"Stack after peek: {stack}")
88     print()
89
90     # Pop items from the stack
91     print("Popping items:")
92     print(f"Popped: {stack.pop()}")
93     print(f"Stack: {stack}")
94     print(f"Popped: {stack.pop()}")
95     print(f"Stack: {stack}")
96     print(f"Popped: {stack.pop()}")
97     print(f"Stack: {stack}")
98     print()
99
100     # Test is_empty after popping all items
101     print(f"Is stack empty? {stack.is_empty()}")
102     print(f"Stack size: {stack.size()}")

```

Output:

```

Is stack empty? True
Stack size: 0

Pushing items: 10, 20, 30
Stack: Stack([10, 20, 30])
Is stack empty? False
Stack size: 3

Peek at top: 30
Stack after peek: Stack([10, 20, 30])

Popping items:
Popped: 30
Stack: Stack([10, 20])
Popped: 20
Stack: Stack([10])
Popped: 10
Stack: Stack([])

Is stack empty? True
Stack size: 0

```

Task-2–Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

Pass

Prompt: complete the queue class within enqueue, dequeue, peek, and size methods and code.

```

1 class Queue:
2     def __init__(self):
3         """Initialize an empty queue."""
4         self.items = []
5
6     def enqueue(self, item):
7         """Add an item to the rear of the queue.
8
9         Args:
10             item: The item to add to the queue
11         """
12         self.items.append(item)
13
14     def dequeue(self):
15         """Remove and return the front item from the queue.
16
17         Returns:
18             The front item of the queue
19
20         Raises:
21             IndexError: If the queue is empty
22         """
23         if self.is_empty():
24             raise IndexError("Cannot dequeue from an empty queue")
25         return self.items.pop(0)
26
27     def peek(self):
28         """Return the front item of the queue without removing it.
29
30         Returns:
31             The front item of the queue
32
33         Raises:
34             IndexError: If the queue is empty
35         """
36         if self.is_empty():
37             raise IndexError("Cannot peek at an empty queue")
38         return self.items[0]
39
40     def size(self):
41         """Return the number of items in the queue.
42
43         Returns:
44             The number of items in the queue
45         """
46         return len(self.items)
47
48     def is_empty(self):
49         """Check if the queue is empty.
50
51         Returns:
52             True if the queue is empty, False otherwise
53         """
54         return len(self.items) == 0
55
56 # Example usage and testing
57 if __name__ == "__main__":
58     # Create a new queue
59     q = Queue()
60
61     # Test enqueue
62     print("Enqueuing items: 1, 2, 3, 4, 5")
63     q.enqueue(1)
64     q.enqueue(2)
65     q.enqueue(3)
66     q.enqueue(4)
67     q.enqueue(5)
68
69     # Test size
70     print(f"Queue size: {q.size()}")
71
72     # Test peek
73     print(f"Peek at front: {q.peek()}")
74
75     # Test dequeue
76     print("\nDequeuing items:")
77     while not q.is_empty():
78         print(f"Dequeued: {q.dequeue()}, Remaining size: {q.size()}")
79
80     # Test empty queue
81     print(f"\nQueue is empty: {q.is_empty()}")
82
83     # Test error handling
84     try:
85         q.dequeue()
86     except IndexError as e:
87         print(f"Error caught: {e}")
88
89     try:
90         q.peek()
91     except IndexError as e:
92         print(f"Error caught: {e}")
93

```

Output:

```

Enqueueing items: 1, 2, 3, 4, 5
Queue size: 5
Peek at front: 1

Dequeueing items:
Dequeued: 1, Remaining size: 4
Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Enqueueing items: 1, 2, 3, 4, 5
Queue size: 5
Peek at front: 1

Dequeueing items:
Dequeued: 1, Remaining size: 4
Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Peek at front: 1

Dequeueing items:
Dequeued: 1, Remaining size: 4
Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Dequeueing items:
Dequeued: 1, Remaining size: 4
Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Dequeueing items:
Dequeued: 1, Remaining size: 4
Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Dequeued: 2, Remaining size: 3
Dequeued: 3, Remaining size: 2
Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Dequeued: 4, Remaining size: 1
Dequeued: 5, Remaining size: 0

Dequeued: 5, Remaining size: 0

Queue is empty: True
Error caught: Cannot dequeue from an empty queue
Error caught: Cannot peek at an empty queue

```

Task-3–LinkedList

Task: Use AI to generate a Singly LinkedList with insert and display methods.

Sample Input Code:

class Node:

Pass

Prompt: generate a Singly Linked List within insert and display methods with code.

Code:

```

1 class Node:
2     """Node class to represent a single node in the linked list"""
3     def __init__(self, data):
4         self.data = data # Data stored in the node
5         self.next = None # Reference to the next node
6
7
8 class SinglyLinkedList:
9     """Singly Linked List implementation with insert and display methods"""
10
11     def __init__(self):
12         self.head = None # Head pointer pointing to the first node
13
14     def insert(self, data):
15         """
16         Insert a new node at the end of the linked list
17
18         Args:
19             data: The data to be inserted into the linked list
20         """
21         new_node = Node(data)
22
23         # If the list is empty, make the new node the head
24         if self.head is None:
25             self.head = new_node
26         else:
27             # Traverse to the end of the list
28             current = self.head
29             while current.next is not None:
30                 current = current.next
31             # Insert the new node at the end
32             current.next = new_node
33
34     def insert_at_beginning(self, data):
35         """
36         Insert a new node at the beginning of the linked list
37
38         Args:
39             data: The data to be inserted into the linked list
40         """
41         new_node = Node(data)
42         new_node.next = self.head
43         self.head = new_node
44
45     def display(self):
46         """
47         Display all elements in the linked list
48         """
49         if self.head is None:
50             print("Linked list is empty")
51             return
52
53         current = self.head
54         elements = []
55         while current is not None:
56             elements.append(str(current.data))
57             current = current.next
58
59         # Display in format: data1 -> data2 -> data3 -> None
60         print(" -> ".join(elements) + " -> None")
61
62
63 # Example usage
64 if __name__ == "__main__":
65     # Create a new linked list
66     ll = SinglyLinkedList()
67
68     # Insert some elements
69     print("Inserting elements into the linked list...")
70     ll.insert(10)
71     ll.insert(20)
72     ll.insert(30)
73     ll.insert(40)
74
75     # Display the linked list
76     print("\nLinked list contents:")
77     ll.display()
78
79     # Insert at beginning
80     print("\nInserting 5 at the beginning...")
81     ll.insert_at_beginning(5)
82     ll.display()
83
84     # Create an empty list
85     print("\nCreating an empty linked list:")
86     empty_ll = SinglyLinkedList()
87     empty_ll.display()

```

Output:

```
Inserting elements into the linked list...
Inserting elements into the linked list...
```

```
Linked List contents:
```

```
Linked List contents:
```

```
Linked List contents:
```

```
10 -> 20 -> 30 -> 40 -> None
```

```
10 -> 20 -> 30 -> 40 -> None
```

```
Inserting 5 at the beginning...
```

```
Inserting 5 at the beginning...
```

```
5 -> 10 -> 20 -> 30 -> 40 -> None
```

```
5 -> 10 -> 20 -> 30 -> 40 -> None
```

```
Creating an empty linked list:
```

```
Linked List is empty
```

```
Linked List is empty
```

Task-4–BinarySearch Tree (BST)

Task: Use AI to create a BST withinsert and in-order traversal methods.

SampleInput Code:

```
classBS
```

```
    T: pass
```

Prompt: create a BST withinsert and in-order traversal methods and code.

Code:

```

1 class Node:
2     def __init__(self, key: int):
3         self.key = key
4         self.left: "Node | None" = None
5         self.right: "Node | None" = None
6
7
8 class BST:
9     def __init__(self):
10         self.root: Node | None = None
11
12     def insert(self, key: int) -> None:
13         """Insert key into the BST (duplicates go to the right)."""
14         if self.root is None:
15             self.root = Node(key)
16             return
17
18         cur = self.root
19         while True:
20             if key < cur.key:
21                 if cur.left is None:
22                     cur.left = Node(key)
23                     return
24                 cur = cur.left
25             else:
26                 if cur.right is None:
27                     cur.right = Node(key)
28                     return
29                 cur = cur.right
30
31     def inorder(self) -> list[int]:
32         """Return keys in in-order (sorted) order."""
33         result: list[int] = []
34
35         def dfs(n: Node | None) -> None:
36             if n is None:
37                 return
38             dfs(n.left)
39             result.append(n.key)
40             dfs(n.right)
41
42         dfs(self.root)
43         return result
44
45
46 if __name__ == "__main__":
47     bst = BST()
48     for x in [7, 3, 9, 1, 5, 8, 10]:
49         bst.insert(x)
50     print("In-order:", bst.inorder())

```

Output:

```
In-order: [1, 3, 5, 7, 8, 9, 10]
```

Task-5–HashTable

Task: Use AI to implement a hashtable with basic insert, search, and delete methods.

Sample Input Code:

Prompt: implement a hashtable with basic insert, search, and delete methods with code.

Code:

```

1 class HashTable:
2     """
3     Hash table using separate chaining (list of buckets).
4
5     Methods:
6     - insert(key, value): add/update a key
7     - search(key): return value or None if not found
8     - delete(key): remove key, return True if removed else False
9     """
10
11 def __init__(self, capacity: int = 8) -> None:
12     if capacity < 1:
13         raise ValueError("capacity must be >= 1")
14     self._capacity = capacity
15     self._buckets = [[] for _ in range(self._capacity)] # List[List[tuple[key, value]]]
16     self._size = 0
17
18 def _index(self, key) -> int:
19     return hash(key) % self._capacity
20
21 def _rehash(self, new_capacity: int) -> None:
22     old_items = []
23     for bucket in self._buckets:
24         old_items.extend(bucket)
25
26     self._capacity = new_capacity
27     self._buckets = [[] for _ in range(self._capacity)]
28     self._size = 0
29
30     for k, v in old_items:
31         self.insert(k, v)
32
33 def insert(self, key, value) -> None:
34     # Resize when load factor gets too high (simple rule-of-thumb)
35     if (self._size + 1) / self._capacity > 0.75:
36         self._rehash(self._capacity * 2)
37
38     idx = self._index(key)
39     bucket = self._buckets[idx]
40
41     for i, (k, _) in enumerate[Any](bucket):
42         if k == key:
43             bucket[i] = (key, value) # update existing
44             return
45
46     bucket.append((key, value))
47     self._size += 1
48
49 def search(self, key):
50     idx = self._index(key)
51     bucket = self._buckets[idx]
52     for k, v in bucket:
53         if k == key:
54             return v
55     return None
56
57 def delete(self, key) -> bool:
58     idx = self._index(key)
59     bucket = self._buckets[idx]
60
61     for i, (k, _) in enumerate[Any](bucket):
62         if k == key:
63             bucket.pop(i)
64             self._size -= 1
65             return True
66
67     return False
68
69 def __len__(self) -> int:
70     return self._size
71
72 def __contains__(self, key) -> bool:
73     return self.search(key) is not None
74
75 def __repr__(self) -> str:
76     return f"HashTable(size={self._size}, capacity={self._capacity})"
77
78
79 if __name__ == "__main__":
80     ht = HashTable()
81     ht.insert("name", "Alice")
82     ht.insert("age", 20)
83     ht.insert("age", 21) # update
84
85     print(ht) # HashTable(...)
86     print(ht.search("name")) # Alice
87     print(ht.search("age")) # 21
88     print(ht.search("x")) # None
89
90     print(ht.delete("age")) # True
91     print(ht.delete("age")) # False
92     print(len(ht)) # 1

```

Output:

```

HashTable(size=2, capacity=8)
Alice
21
None
HashTable(size=2, capacity=8)
Alice
21
None
21
None
True
False
1
True
False
1
False
1

```

Task-6–Graph Representation

Task: Use AI to implement a graph using an adjacency list. Sample **Input**

Code:

Class Graph:

pass

Prompt:

implement a graph using an adjacency list with code

Code:

```

1 class Graph:
2     """
3     Graph implemented using an adjacency list.
4
5     - By default the graph is undirected.
6     - Set directed=True for a directed graph.
7     """
8
9     def __init__(self, directed: bool = False):
10         self.directed = directed
11         # adjacency list: vertex -> set of neighbor vertices
12         self.adj: dict[object, set[object]] = {}
13
14     def add_vertex(self, v: object) -> None:
15         """Add a vertex if it doesn't already exist."""
16         if v not in self.adj:
17             self.adj[v] = set[object]()
18
19     def add_edge(self, u: object, v: object) -> None:
20         """Add an edge u -> v (and v -> u if undirected)."""
21         self.add_vertex(u)
22         self.add_vertex(v)
23         self.adj[u].add(v)
24         if not self.directed:
25             self.adj[v].add(u)
26
27     def remove_edge(self, u: object, v: object) -> None:
28         """Remove an edge u -> v (and v -> u if undirected), if present."""
29         if u in self.adj:
30             self.adj[u].discard(v)
31         if not self.directed and v in self.adj:
32             self.adj[v].discard(u)
33
34     def remove_vertex(self, v: object) -> None:
35         """Remove a vertex and all edges incident to it."""
36         if v not in self.adj:
37             return
38
39         # Remove edges from neighbors to v
40         for n in list(self.adj[v]):
41             self.remove_edge(v, n)
42
43         # In directed graphs, also remove incoming edges to v
44         if self.directed:
45             for u in self.adj:
46                 self.adj[u].discard(v)
47
48         del self.adj[v]
49
50     def neighbors(self, v: object) -> list[object]:
51         """Return neighbors of v as a sorted list when possible."""
52         if v not in self.adj:
53             return []
54         try:
55             return sorted(self.adj[v])
56         except TypeError:
57             return list(self.adj[v])
58
59     def bfs(self, start: object) -> list[object]:
60         """Breadth-first traversal order starting from start."""
61         if start not in self.adj:
62             return []
63
64         visited = {start}
65         queue = [start]
66         order: list[object] = []
67
68         while queue:
69             v = queue.pop(0)
70             order.append(v)
71             for n in self.neighbors(v):
72                 if n not in visited:
73                     visited.add(n)
74                     queue.append(n)
75
76         return order
77
78     def dfs(self, start: object) -> list[object]:
79         """Depth-first traversal order starting from start."""
80         if start not in self.adj:
81             return []
82
83         visited: set[object] = set[object]()
84         order: list[object] = []
85
86         def _visit(v: object) -> None:
87             visited.add(v)
88             order.append(v)
89             for n in self.neighbors(v):
90                 if n not in visited:
91                     _visit(n)
92
93         _visit(start)
94         return order
95
96     def __str__(self) -> str:
97         lines = []
98         for v in self.adj:
99             lines.append(f"{v} -> {self.neighbors(v)}")
100         return "\n".join(lines)
101
102 if __name__ == "__main__":
103     g = Graph(directed=False) # change to True for a directed graph
104     g.add_edge("A", "B")
105     g.add_edge("A", "C")
106     g.add_edge("B", "D")
107     g.add_edge("C", "D")
108     g.add_edge("D", "E")
109
110     print("Adjacency list:")
111     print(g)
112     print()
113     print("BFS from A:", g.bfs("A"))
114     print("DFS from A:", g.dfs("A"))

```

Output:

```
Adjacency list:
A -> ['B', 'C']
B -> ['A', 'D']
Adjacency list:
A -> ['B', 'C']
B -> ['A', 'D']
A -> ['B', 'C']
B -> ['A', 'D']
B -> ['A', 'D']
C -> ['A', 'D']
D -> ['B', 'C', 'E']
E -> ['D']

BFS from A: ['A', 'B', 'C', 'D', 'E']
DFS from A: ['A', 'B', 'D', 'C', 'E']
D -> ['B', 'C', 'E']
E -> ['D']

BFS from A: ['A', 'B', 'C', 'D', 'E']
DFS from A: ['A', 'B', 'D', 'C', 'E']
E -> ['D']

BFS from A: ['A', 'B', 'C', 'D', 'E']
DFS from A: ['A', 'B', 'D', 'C', 'E']
BFS from A: ['A', 'B', 'C', 'D', 'E']
DFS from A: ['A', 'B', 'D', 'C', 'E']
DFS from A: ['A', 'B', 'D', 'C', 'E']
```

Task-7–Priority Queue

Task: Use A to implement a priority queue using Python's `heapq` module.

Sample Input Code:

Class Priority

Queue: pass

Prompt:

implement a priority queue using Python's `heapq` module with code

Code:

```

1 import heapq
2 from itertools import count
3
4 class PriorityQueue:
5     """
6     Min-priority queue by default (smaller priority value = served first).
7     For max-priority behavior, push with -priority.
8     """
9     def __init__(self):
10         self._heap = []
11         self._seq = count[int]() # tie-breaker for equal priorities (FIFO)
12
13     def push(self, item, priority: int):
14         heapq.heappush(self._heap, (priority, next(self._seq), item))
15
16     def pop(self):
17         if not self._heap:
18             raise IndexError("pop from empty PriorityQueue")
19         priority, _, item = heapq.heappop(self._heap)
20         return item, priority
21
22     def peek(self):
23         if not self._heap:
24             raise IndexError("peek from empty PriorityQueue")
25         priority, _, item = self._heap[0]
26         return item, priority
27
28     def __len__(self):
29         return len(self._heap)
30
31     def empty(self):
32         return len(self._heap) == 0
33
34 if __name__ == "__main__":
35     pq = PriorityQueue()
36     pq.push("low", 5)
37     pq.push("urgent", 1)
38     pq.push("medium", 3)
39     pq.push("also urgent (arrives later)", 1)
40
41     while not pq.empty():
42         item, pr = pq.pop()
43         print(pr, item)
44
45     # Max-priority example (bigger number = served first):
46     maxpq = PriorityQueue()
47     for item, pr in [("A", 10), ("B", 2), ("C", 10)]:
48         maxpq.push(item, -pr) # negate priority
49
50     print("max first:", maxpq.pop()) # returns (item, neg_priority)
51

```

Output:

```

1 urgent
1 also urgent (arrives later)
3 medium
5 low
max first: ('A', -10)

```

Task-8–Deque

Task: Use `collections.deque` to implement a double-ended queue.

Sample Input Code:

Class

DequeDS:

pass

Prompt:

implement a double-ended queue using `collections.deque` with code

Code:

```

1  from __future__ import annotations
2
3  from collections import deque
4  from typing import Deque, Generic, Iterator, Optional, TypeVar
5
6  T = TypeVar("T")
7
8
9  class DequeDS(Generic[T]):
10     """
11     Double-ended queue (deque) implemented using collections.deque.
12     Supports O(1) append/pop operations on both ends.
13     """
14
15     def __init__(self, items: Optional[Iterator[T]] = None) -> None:
16         self._dq: Deque[T] = deque[T](items or [])
17
18     # --- Add operations ---
19     def add_front(self, item: T) -> None:
20         """Insert item at the front (left)."""
21         self._dq.appendleft(item)
22
23     def add_rear(self, item: T) -> None:
24         """Insert item at the rear (right)."""
25         self._dq.append(item)
26
27     # --- Remove operations ---
28     def remove_front(self) -> T:
29         """Remove and return the front (left) item."""
30         if self.is_empty():
31             raise IndexError("remove_front from empty deque")
32         return self._dq.popleft()
33
34     def remove_rear(self) -> T:
35         """Remove and return the rear (right) item."""
36         if self.is_empty():
37             raise IndexError("remove_rear from empty deque")
38         return self._dq.pop()
39
40     # --- Peek operations ---
41     def peek_front(self) -> T:
42         """Return the front (left) item without removing it."""
43         if self.is_empty():
44             raise IndexError("peek_front from empty deque")
45         return self._dq[0]
46
47     def peek_rear(self) -> T:
48         """Return the rear (right) item without removing it."""
49         if self.is_empty():
50             raise IndexError("peek_rear from empty deque")
51         return self._dq[-1]
52
53     # --- Utility ---
54     def is_empty(self) -> bool:
55         return len(self._dq) == 0
56
57     def size(self) -> int:
58         return len(self._dq)
59
60     def clear(self) -> None:
61         self._dq.clear()
62
63     def __len__(self) -> int:
64         return len(self._dq)
65
66     def __iter__(self) -> Iterator[T]:
67         return iter(self._dq)
68
69     def __repr__(self) -> str:
70         return f"DequeDS({list[T](self._dq)!r})"
71
72
73  if __name__ == "__main__":
74      d = DequeDS[int]()
75      d.add_front(10)    # [10]
76      d.add_rear(20)     # [10, 20]
77      d.add_front(5)     # [5, 10, 20]
78      print("Deque:", d)
79      print("Front:", d.peek_front())
80      print("Rear:", d.peek_rear())
81      print("Remove front:", d.remove_front()) # 5
82      print("Remove rear:", d.remove_rear())   # 20
83      print("Deque now:", d)

```

Output:

```
Deque: DequeDS([5, 10, 20])
Front: 5
Rear: 20
Remove front: 5
Remove rear: 20
Deque now: DequeDS([10])
```

Task-9 Real-Time Application Challenge—Choose the Right Data Structure

Prompt:

Solve this clearly and concisely.

Design a Campus Resource Management System code with:

1. Student Attendance Tracking
2. Event Registration System
3. Library Book Borrowing
4. Bus Scheduling System
5. Cafeteria Order Queue

Choose the best data structure for each feature from:

Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque

Output as a table:

Feature | Data Structure | 2–3 sentence justification

Code:

[illegible]

[illegible]

Output:

```

Attendance S1 %: 50.0
Confirmed E100: ['S1', 'S2']
Waitlist E100: ['S3']
After cancel S2, confirmed: ['S1', 'S3']
Borrow Fluent Python: True
Borrow Fluent Python again: False
Catalog in order: [('9780134685991', 2), ('9781492051367', 0)]
Shortest bus path Hostel->Library: ['Hostel', 'Cafeteria', 'Library'] minutes: 9
Attendance S1 %: 50.0
Confirmed E100: ['S1', 'S2']
Waitlist E100: ['S3']
After cancel S2, confirmed: ['S1', 'S3']
Borrow Fluent Python: True
Borrow Fluent Python again: False
Catalog in order: [('9780134685991', 2), ('9781492051367', 0)]
Shortest bus path Hostel->Library: ['Hostel', 'Cafeteria', 'Library'] minutes: 9
Waitlist E100: ['S3']
After cancel S2, confirmed: ['S1', 'S3']
Borrow Fluent Python: True
Borrow Fluent Python again: False
Catalog in order: [('9780134685991', 2), ('9781492051367', 0)]
Shortest bus path Hostel->Library: ['Hostel', 'Cafeteria', 'Library'] minutes: 9
Borrow Fluent Python again: False
Catalog in order: [('9780134685991', 2), ('9781492051367', 0)]
Shortest bus path Hostel->Library: ['Hostel', 'Cafeteria', 'Library'] minutes: 9
Serve order: CafeteriaOrder(order_id=2, student_id='S2', item='Coffee', priority=2)
Shortest bus path Hostel->Library: ['Hostel', 'Cafeteria', 'Library'] minutes: 9
Serve order: CafeteriaOrder(order_id=2, student_id='S2', item='Coffee', priority=2)
Serve order: CafeteriaOrder(order_id=2, student_id='S2', item='Coffee', priority=2)
Serve order: CafeteriaOrder(order_id=3, student_id='S3', item='Burger', priority=1)
Serve order: CafeteriaOrder(order_id=3, student_id='S3', item='Burger', priority=1)

```

Task-10: SmartE-Commerce Platform – Data Structure Prompt:

Solve this clearly and concisely.

Design a SmartE-Commerce Platform with:

Shopping Cart Management – Add/remove products dynamically

Order Processing System – Process orders in placement order

Top-Selling Products Tracker – Rank products by sales count

Product Search Engine – Fast lookup using product ID

Delivery Route Planning – Connect warehouses and delivery locations

Choose the most appropriate data structure for each feature from:

Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque

Output as a table:

Feature | Data Structure | 2–3 sentence justification

Code:

```

1 from collections import deque
2 import heapq
3 from typing import Dict, List, Tuple, Optional
4
5 # -----
6 # Product model
7 # -----
8
9 class Product:
10     def __init__(self, product_id: int, name: str, price: float):
11         self.id = product_id
12         self.name = name
13         self.price = price
14
15     def __repr__(self):
16         return f"Product(id={self.id}, name='{self.name}', price={self.price})"
17
18 # -----
19 # Product Search Engine (Hash Table)
20 # -----
21
22 class ProductSearchEngine:
23     def __init__(self):
24         # Hash Table: product_id -> Product
25         self.products: Dict[int, Product] = {}
26
27     def add_product(self, product: Product):
28         self.products[product.id] = product
29
30     def get_product(self, product_id: int) -> Optional[Product]:
31         return self.products.get(product_id)
32
33     def remove_product(self, product_id: int):
34         self.products.pop(product_id, None)
35
36 # -----
37 # Shopping Cart (Linked List)
38 # -----
39
40 class CartNode:
41     def __init__(self, product: Product, quantity: int):
42         self.product = product
43         self.quantity = quantity
44         self.next: Optional[CartNode] = None
45
46 class ShoppingCart:
47     def __init__(self):
48         self.head: Optional[CartNode] = None
49
50     def add_product(self, product: Product, quantity: int = 1):
51         """
52         If product already exists in the list, increase quantity.
53         Otherwise, add new node at the front (O(1) insertion).
54         """
55         new_node = CartNode(product, quantity)
56         while self.head:
57             if self.head.product.id == product.id:
58                 self.head.quantity += quantity
59                 return
60             self.head = self.head.next
61         new_node.next = self.head
62         self.head = new_node
63
64     def remove_product(self, product_id: int, quantity: int = None):
65         """
66         Remove some or all quantity of a product.
67         If quantity is None or reaches 0, remove the node.
68         """
69         prev = None
70         node = self.head
71         while node:
72             if node.product.id == product_id:
73                 if quantity is None or node.quantity <= quantity:
74                     # delete the node
75                     if prev:
76                         prev.next = node.next
77                     else:
78                         self.head = node.next
79                 else:
80                     node.quantity -= quantity
81                     return
82                 prev = node
83                 node = node.next
84             else:
85                 prev = node
86                 node = node.next
87
88     def list_items(self) -> List[Tuple[Product, int]]:
89         result = []
90         node = self.head
91         while node:
92             result.append((node.product, node.quantity))
93             node = node.next
94         return result
95
96     def total_price(self) -> float:
97         return sum(node.product.price * node.quantity
98                     for node in self._iter_nodes())
99
100     def _iter_nodes(self):
101         node = self.head
102         while node:
103             yield node
104             node = node.next
105
106 # -----
107 # Order Processing System (Queue)
108 # -----
109
110 class Order:
111     __next_id = 1
112
113     def __init__(self, cart_snapshot: List[Tuple[Product, int]]):
114         self.id = Order.__next_id
115         Order.__next_id += 1
116         self.items = cart_snapshot # list of (Product, quantity)
117
118     def __repr__(self):
119         return f"Order(id={self.id}, items=[{(p.id, q) for p, q in self.items}])"
120
121
122 class OrderProcessingSystem:
123     def __init__(self):
124         # Queue of orders (FIFO)
125         self.queue: deque[Order] = deque[Order]()
126
127     def place_order(self, cart: ShoppingCart) -> Order:
128         order = Order(cart.list_items())
129         self.queue.append(order)
130         return order
131
132     def process_next_order(self) -> Optional[Order]:
133         if not self.queue:
134             return None
135         return self.queue.popleft()
136
137     def pending_orders(self) -> int:
138         return len(self.queue)
139
140 # -----
141 # Top-Selling Products Tracker (Priority Queue / Min-Heap)
142 # -----
143
144 class TopSellingProductsTracker:
145     def __init__(self):
146         # product_id -> sales_count
147         self.sales: Dict[int, int] = {}
148         # priority queue entries: (-sales_count, product_id)
149         self.heap: List[Tuple[int, int]] = []
150
151     def record_sale(self, product_id: int, quantity: int = 1):
152         self.sales[product_id] = self.sales.get(product_id, 0) + quantity
153         # Push new priority entry; lazy update (we'll sort+pop next time sales on pop)
154         heapq.heappush(self.heap, (-self.sales[product_id], product_id))
155
156     def top_k(self, k: int) -> List[Tuple[int, int]]:
157         """
158         Returns list of (product_id, sales_count) for top k products.
159         Uses lazy removal from the heap to keep it consistent.
160         """

```

```

152     result = []
153     seen = set[any]()
154
155     while self.heap and len(result) < k:
156         neg_sales, pid = heapq.heappop(self.heap)
157         current_sales = self.sales.get(pid, 0)
158
159         if current_sales == -neg_sales and pid not in seen:
160             result.append(pid, current_sales)
161             seen.add(pid)
162
163         # push back the elements we popped that are still valid
164         for pid in seen:
165             heapq.heappush(self.heap, (-self.sales[pid], pid))
166
167     return result
168
169 # -----
170 # Delivery Route Planning (Graph + Dijkstra)
171 # -----
172 class DeliveryRoutePlanner:
173     def __init__(self):
174         # Graph as adjacency list: node -> list of (neighbor, distance)
175         self.graph: Dict[str, List[Tuple[str, float]]] = {}
176
177     def add_location(self, name: str):
178         if name not in self.graph:
179             self.graph[name] = []
180
181     def add_route(self, from_loc: str, to_loc: str, distance: float, bidirectional: bool = True):
182         self.add_location(from_loc)
183         self.add_location(to_loc)
184         self.graph[from_loc].append((to_loc, distance))
185         if bidirectional:
186             self.graph[to_loc].append((from_loc, distance))
187
188     def shortest_path(self, start: str, end: str) -> Tuple[float, List[str]]:
189         """
190         Dijkstra's algorithm: returns (distance, path).
191         Distance is float('inf') if no path exists.
192         """
193         if start not in self.graph or end not in self.graph:
194             return float('inf'), []
195
196         # min-heap: (distance, node, path)
197         heap = [(0.0, start, [start])]
198         visited = set[any]()
199
200         while heap:
201             dist, node, path = heapq.heappop(heap)
202             if node in visited:
203                 continue
204             visited.add(node)
205
206             if node == end:
207                 return dist, path
208
209             for neighbor, weight in self.graph[node]:
210                 if neighbor not in visited:
211                     heapq.heappush(heap, (dist + weight, neighbor, path + [neighbor]))
212
213         return float('inf'), []
214
215 # -----
216 # Example usage
217 # -----
218 if __name__ == "__main__":
219     # Product Search Engine
220     search_engine = ProductSearchEngine()
221     p1 = Product(1, "Laptop", 1000.0)
222     p2 = Product(2, "Phone", 500.0)
223     p3 = Product(3, "Headphones", 100.0)
224     for p in (p1, p2, p3):
225         search_engine.add_product(p)
226
227     # Shopping cart
228     cart = ShoppingCart()
229     cart.add_product(search_engine.get_product(1), 1)
230     cart.add_product(search_engine.get_product(2), 2)
231     cart.add_product(search_engine.get_product(3), 1)
232     cart.remove_product(3, 1) # remove 1 headphone
233
234     print("Cart items:", cart.list_items())
235     print("Total price:", cart.total_price())
236
237     # Order processing
238     ops = OrderProcessingSystem()
239     orders = ops.place_order(cart)
240     print("Placed orders:", orders)
241     print("Pending orders:", ops.pending_orders())
242     processed = ops.process_next_order()
243     print("Processed orders:", processed)
244     print("Pending orders:", ops.pending_orders())
245
246     # Top-selling products
247     tracker = TopSellingProductsTracker()
248     tracker.record_sale(1, 10) # Laptop sold 10
249     tracker.record_sale(2, 5) # Phone sold 5
250     tracker.record_sale(3, 7) # Headphones sold 7
251     print("Top 2 products (id, sales):", tracker.top_k(2))
252
253     # Delivery route planner
254     planner = DeliveryRoutePlanner()
255     planner.add_route("WarehouseA", "City1", 10.0)
256     planner.add_route("WarehouseA", "City2", 20.0)
257     planner.add_route("City1", "City2", 5.0)
258     planner.add_route("City2", "City3", 7.0)
259
260     dist, path = planner.shortest_path("WarehouseA", "City3")
261     print("Shortest route WarehouseA -> City3: [", path, "], distance:", dist)

```

Output:

```

Cart items: [(Product(id=3, name='Headphones', price=100.0), 2), (Product(id=2, name='Phone', price=500.0), 2), (Product(id=1, name='Laptop', price=1000.0), 1)]
Total price: 2200.0
Placed order: Order(id=1, items=[(3, 2), (2, 2), (1, 1)])
Pending orders: 1
Processed order: Order(id=1, items=[(3, 2), (2, 2), (1, 1)])
Pending orders: 0
Top 2 products (id, sales): [(1, 10), (3, 7)]
Shortest route WarehouseA -> City3: ['WarehouseA', 'City1', 'City2', 'City3'] distance: 22.0
PS C:\2403A51103\3-2\AI_A_C\Cursor AI>

Total price: 2200.0
Placed order: Order(id=1, items=[(3, 2), (2, 2), (1, 1)])
Pending orders: 1
Processed order: Order(id=1, items=[(3, 2), (2, 2), (1, 1)])
Pending orders: 0
Top 2 products (id, sales): [(1, 10), (3, 7)]
Shortest route WarehouseA -> City3: ['WarehouseA', 'City1', 'City2', 'City3'] distance: 22.0
Shortest route WarehouseA -> City3: ['WarehouseA', 'City1', 'City2', 'City3'] distance: 22.0

```